

Hybrid Computing Technology

1

Hybrid Computing Code Processes

ผศ. อัครินทร์ คุณกิตติ

ASST. PROF. AKHARIN KHUNKITTI

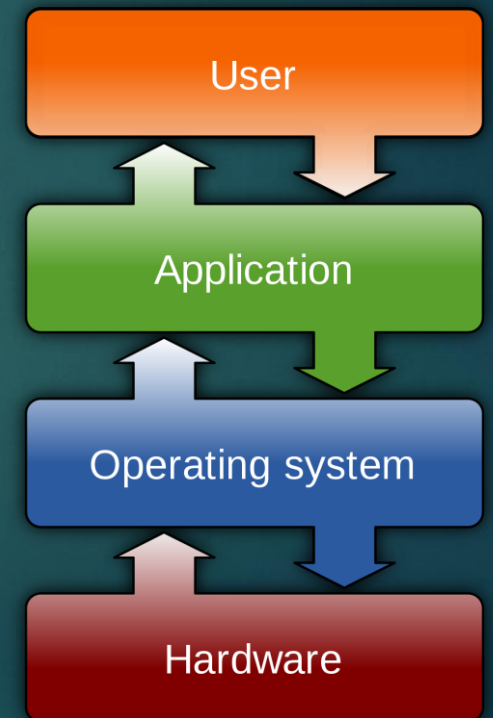
FACULTY OF INFORMATION TECHNOLOGY
KMITL



Topics

2

- ▶ Computing Technology Classification
- ▶ Code and Software Computing Processes
- ▶ Computing Frameworks
- ▶ Conclusion



Computing Technology Classification

3

► Temporal Computing

► CPU – Execute via Instructions

► SISD

► GPU – Execute via Instructions

► SIMD (Massive)

► Spatial Computing

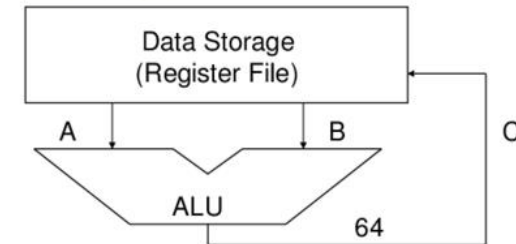
► FPGA – Execute via Direct Logics

► Programmable

► ASIC – Execute via Direct Logics

► Fix

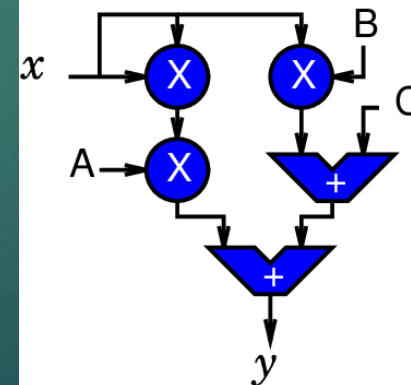
COMPUTING: TEMPORAL



- computing a function through the executing a sequence of instructions in *TIME*
- Generalized to perform many functions well.
- Operates on fixed data sizes.
- Inherently sequential
 - Constrained even with multiple data paths.

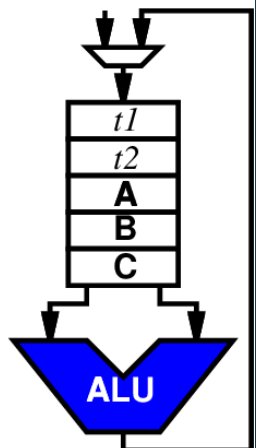
1

Spatial Computation



Temporal Computation

$t1 \leftarrow x$
 $t2 \leftarrow A \times t1$
 $t2 \leftarrow t2 + B$
 $t2 \leftarrow t2 \times t1$
 $y \leftarrow t2 + C$



Computing Technology Classification

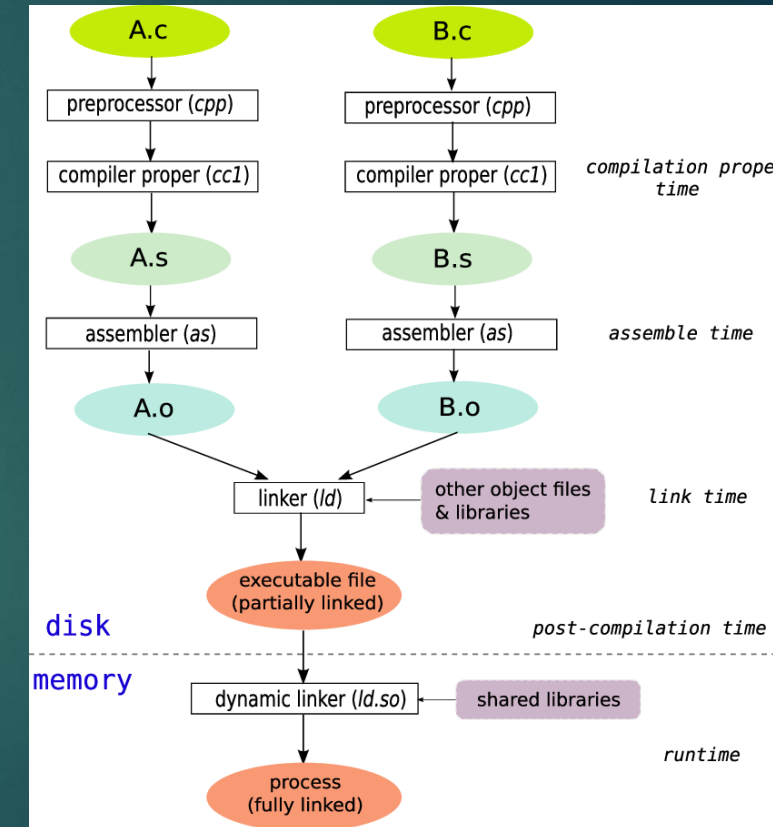
4

Computing Class	Executing Concept	Logic Operation	Re-Configurable	Executing Overhead
1. Temporal Computing	Instruction-Based	Indirect	Yes	High
1.1 CPU	SISD/Single	Inst. Decode	Program	Decoder/Control
1.2 GPU	SIMD/Massive	Inst. Decode	Program	Decoder/Control
2. Spatial Computing	Direct Logic	Direct	May be	Low
2.1 FPGA	Programmable Logic	Direct Config.	Programmable	Config. Logic
2.2 ASIC	Fixed Design Logic	Direct Design	Fixed	Designed Logic

Code Computing Processes

5

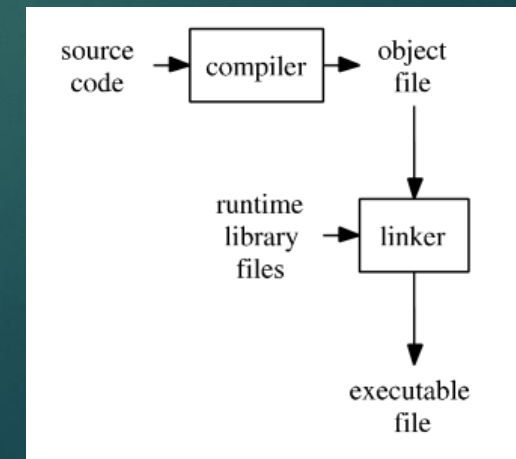
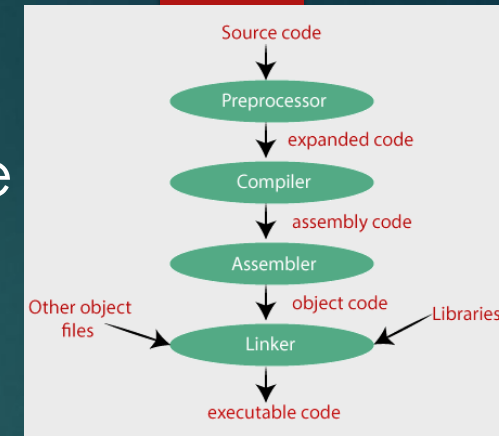
- ▶ Code Computing divided 2 sides
 - ▶ Host(s) – Control and Manage Devices
 - ▶ Device(s) – Execute Codes
- ▶ Code Development Process
 - ▶ Source Codes => Executable Codes
- ▶ Code Execution Process
 - ▶ Executable Code run by Devices' Components/Engines
- ▶ Code Verification Process
 - ▶ Debugging
 - ▶ Verify Code in Development and Execution Processes



Code Development Process

6

- ▶ Source Code – High Level Code / Programming Language
 - ▶ => C/C++, Python, etc.
- ▶ High Level Compiler – Translate to Intermediate Code
- ▶ Intermediate Code – Instruction Code / Hardware Level (HDL)
- ▶ Intermediate Compiler – Translate to Executable Code
- ▶ Linker – Link / Bind / Route / Place Codes into Correct Positions
- ▶ Executable Code – Code to be executed
 - ▶ => Instructions/Configuration Files



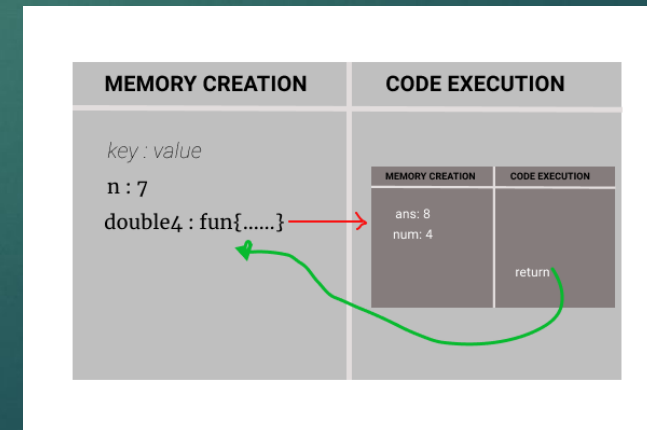
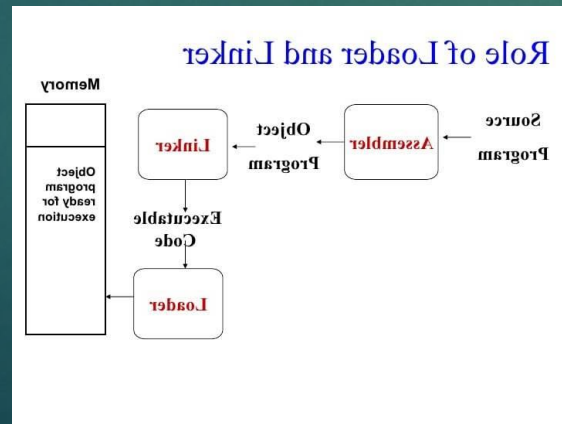
Code Development Process

7

Item	CPU	GPU	FPGA	ASIC
Source Code	Programming Languages (C/C++,Python, etc.)	Programming Languages (C/C++,Python, etc.)	Programming Languages (C/C++,Python, etc.)	Programming Languages (C/C++,Python, etc.)
High Level Compiler	Compiler	Compiler	HLS Tools	HLS Tools
Intermediate Code	Object Code	Object Code	HDL Code (Verilog, VHDL)	HDL Code (Verilog, VHDL)
Intermediate Compiler	Option	Option	HDL Synthesis Tools	HDL Synthesis Tools
Linker	Linker with Libraries	Linker with Libraries	Place / Map / Route Tools	Place / Map / Route Tools
Executable Code	Executable Code / Program	Executable Code / Program	Config. Bit File (Program)	Config. File (Fabrication)

Code Execution Process

- ▶ Executable Code – Instructions / Configuration File
- ▶ Code Loader – Install / Load Code into The Places
- ▶ Code Trigger – Start / Trig Code to be Executed
- ▶ Code Executer – Logic Operation / Running
- ▶ Code Ender – Finish Code Operation
- ▶ Code Un-Loader – Uninstall / Remove Code from the Places



Code Execution Process

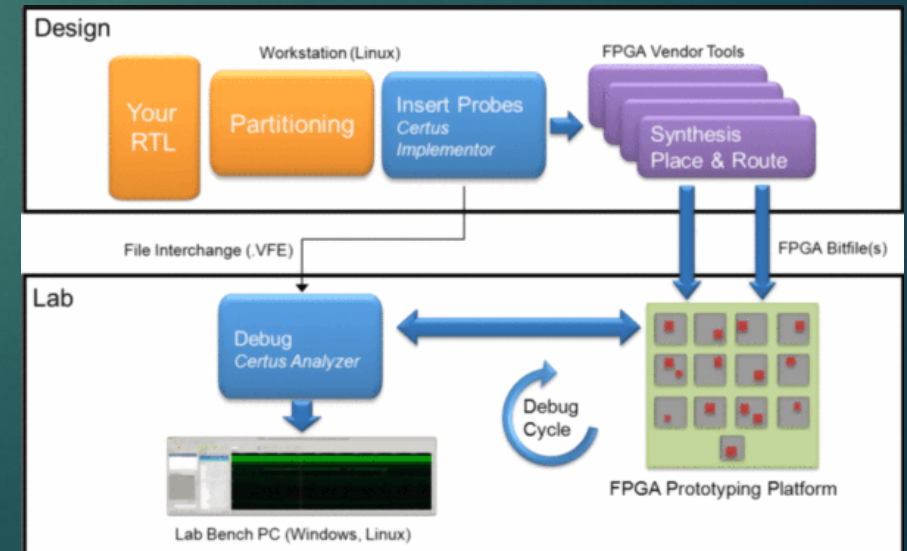
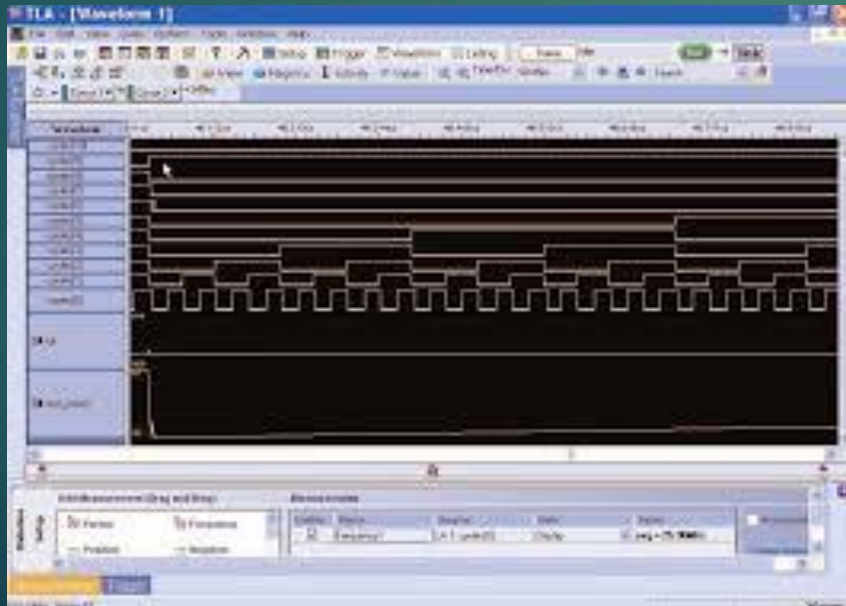
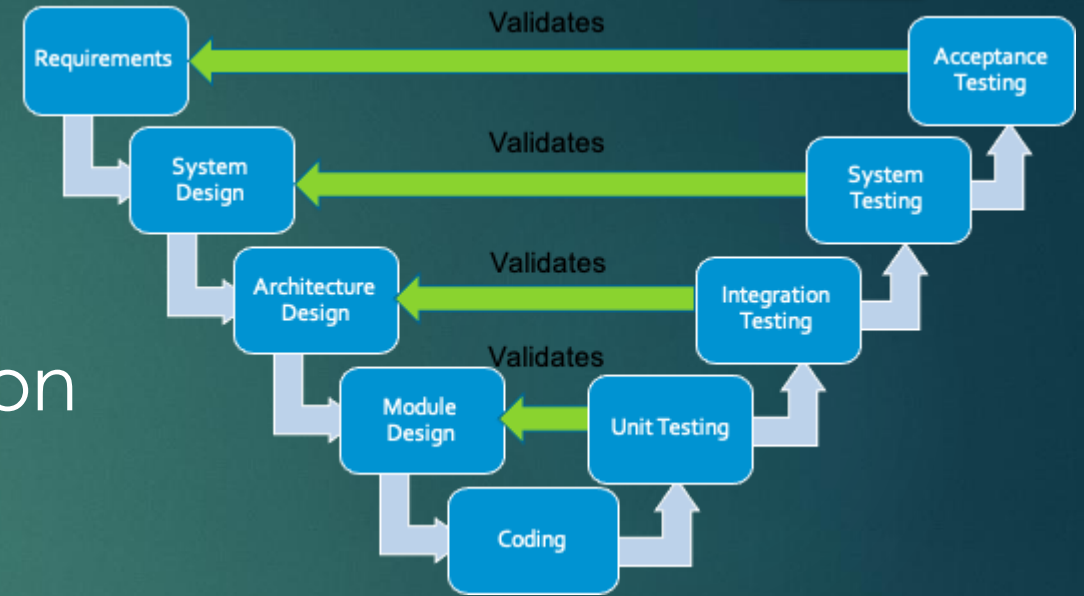
9

Step	CPU	GPU	FPGA	ASIC
Executable Code	Instructions	Instructions	Configuration	Configuration
Code Loader	Into Memory	Into GPU Memory	Device Configuring (Programmed)	Chip Fabrication / Card Assembly
Code Trigger	Function Calling	Function Calling	Start Signal	Start Signal
Code Executer	Instructions Executing	Instructions Executing (Parallel)	Logics Executing	Logics Executing
Code Ender	Function Ending (Return)	Function Ending (Return)	Logic Stages Ending / Streams (Output)	Logic Stages Ending / Streams (Output)
Code Un-Loader	Code Removing / Replacing	Code Removing / Replacing	Config. Removing / Replacing	Cannot Remove (Fixed)

Code Verification Process

10

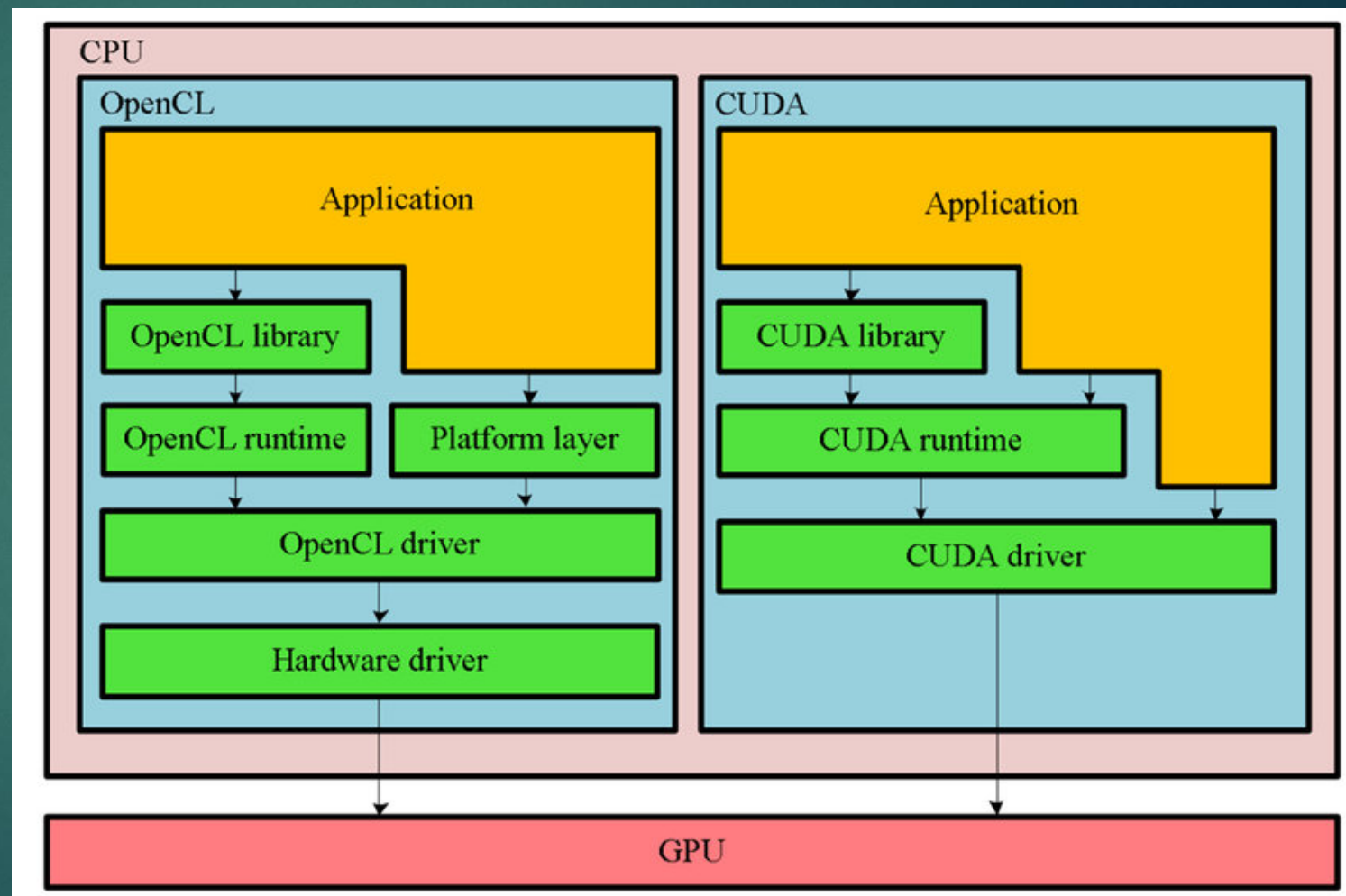
- ▶ Debugger
- ▶ Tester
- ▶ Development Process Verification
- ▶ Execution Process Verification



Computing Frameworks

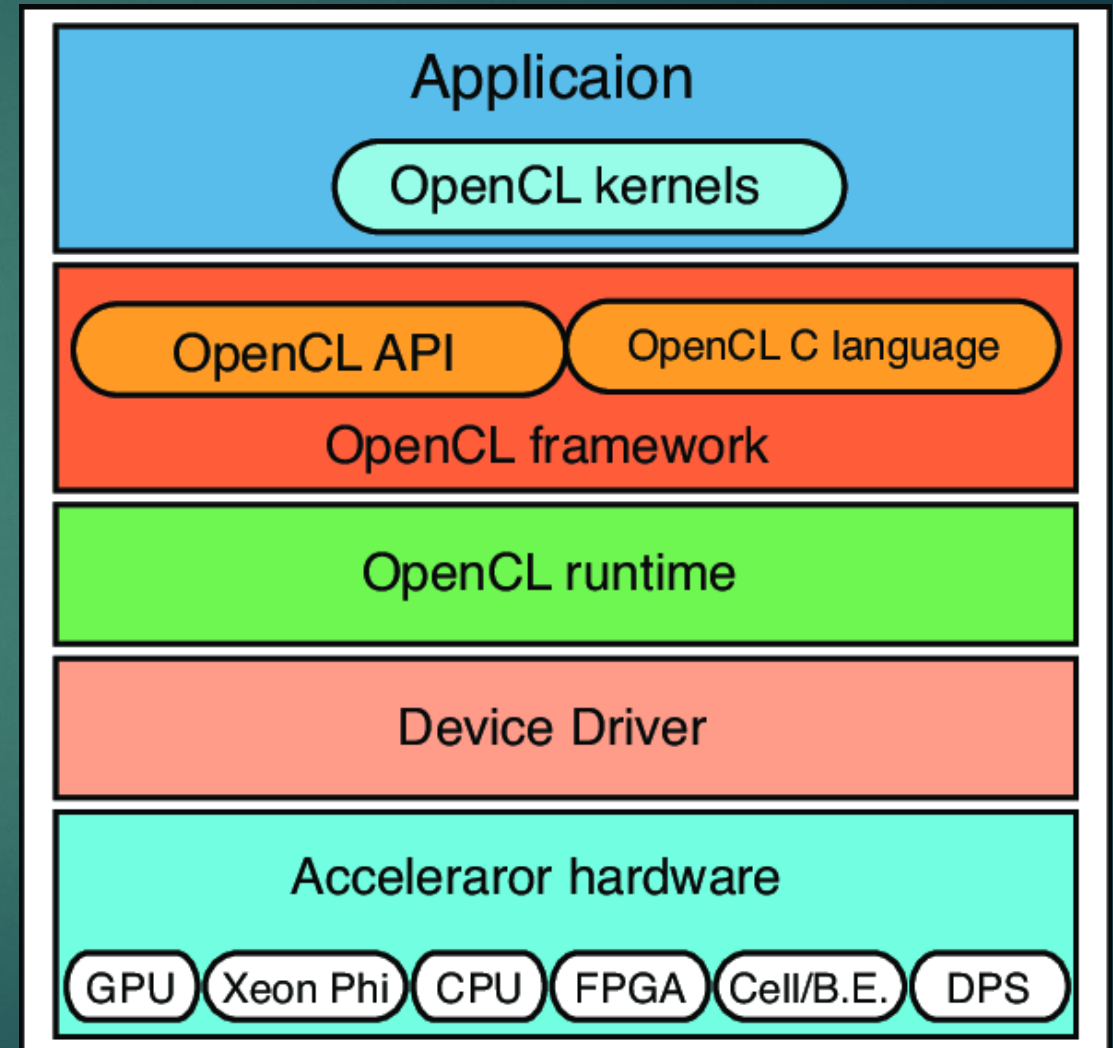
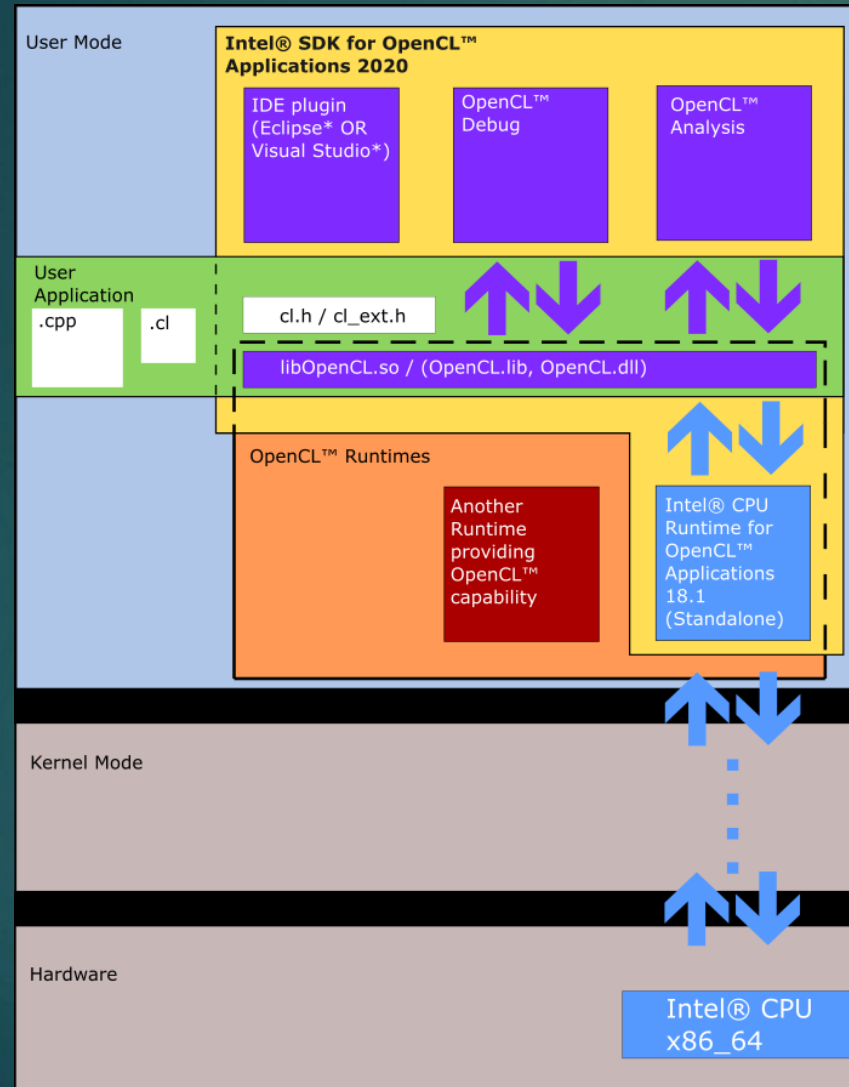
11

- ▶ Host Side
- ▶ Device Sides
 - ▶ CPU
 - ▶ X86/x64
 - ▶ ARM
 - ▶ GPU
 - ▶ NVIDIA
 - ▶ AMD
 - ▶ FPGA
 - ▶ Xilinx
 - ▶ Intel (Altera)
 - ▶ ASIC/TPU



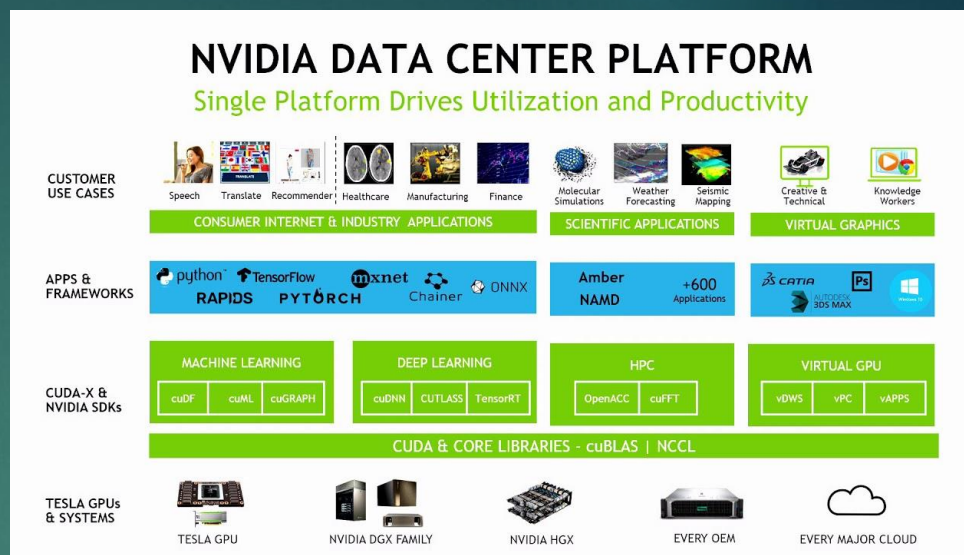
CPU Framework

12

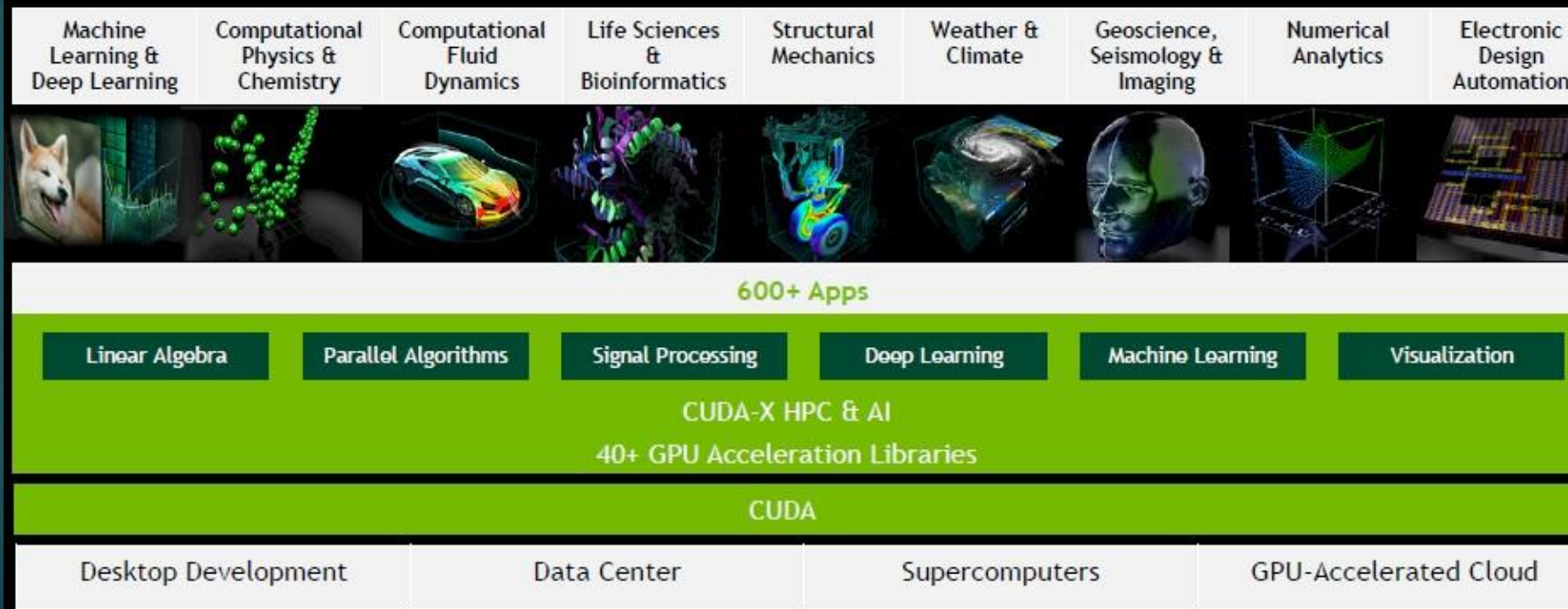


NVIDIA Framework

13



Software To Deliver Acceleration For HPC & AI Apps; 500+ New Updates



AMD Framework

REVOLUTION IN GPU COMPUTING

Radeon Open Compute Platform (ROCm)

Modern Heterogeneous HPC and Hyper Scale Accelerator Platform for Large Scale Systems



Performance

Rich foundation built for latency reduction and throughput optimization



Open

First fully "Open Source" professional GPU accelerator computing solution



Hyper Scale

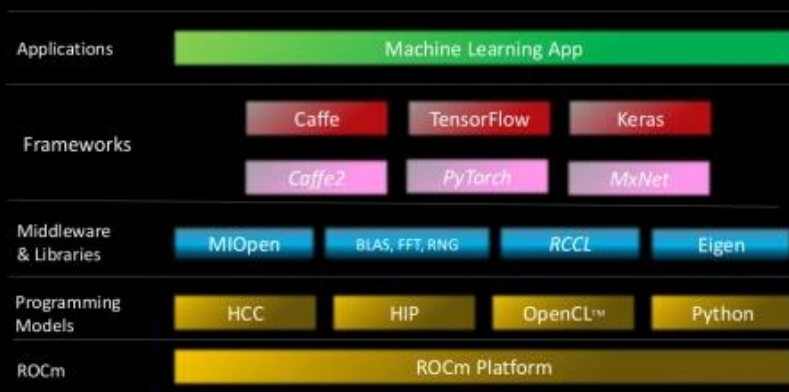
Built from the ground up to service multi-accelerators in node and across the rack

Applications	HPC & Machine Learning Apps			
Tools	Debugger	Performance Analysis	System Validation	System Management
Frameworks	TensorFlow	PyTorch	Kokkos	RAJA
Math Libraries	MIOpen	FFT, RNG	BLAS, Sparse	Eigen
Communication Libraries	RCCL	UCX, libfabric	MPICH	OpenMPI
Programming Models with ROCm	OpenMP	HIP	OpenCL™	Python
Fully Open Source ROCm Platform				
Devices	GPU	CPU	APU	DLA

14



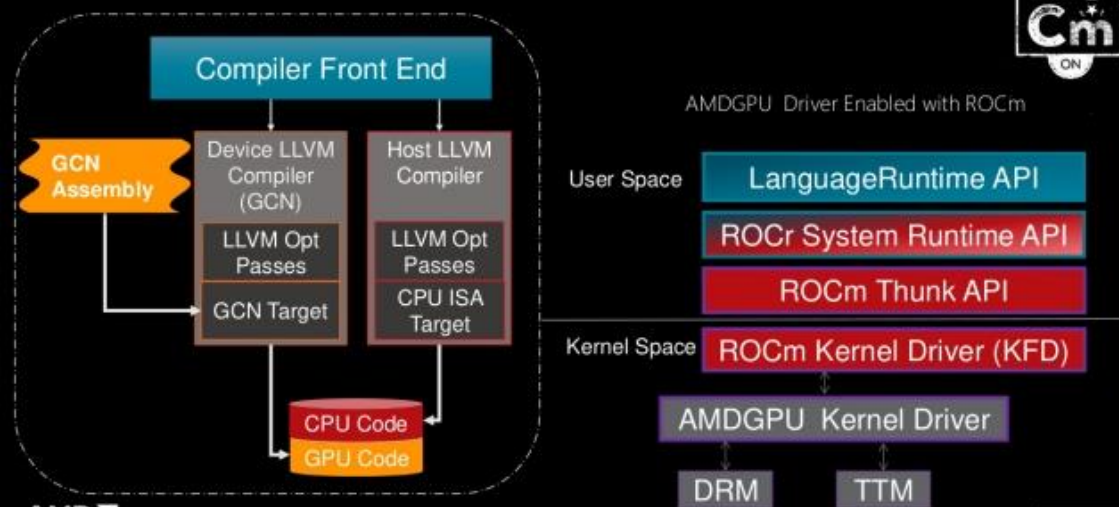
ROCm ML SOFTWARE STACK



Italics = Under Dev

AMD | RADEON

ROCm PLATFORM ON LINUX



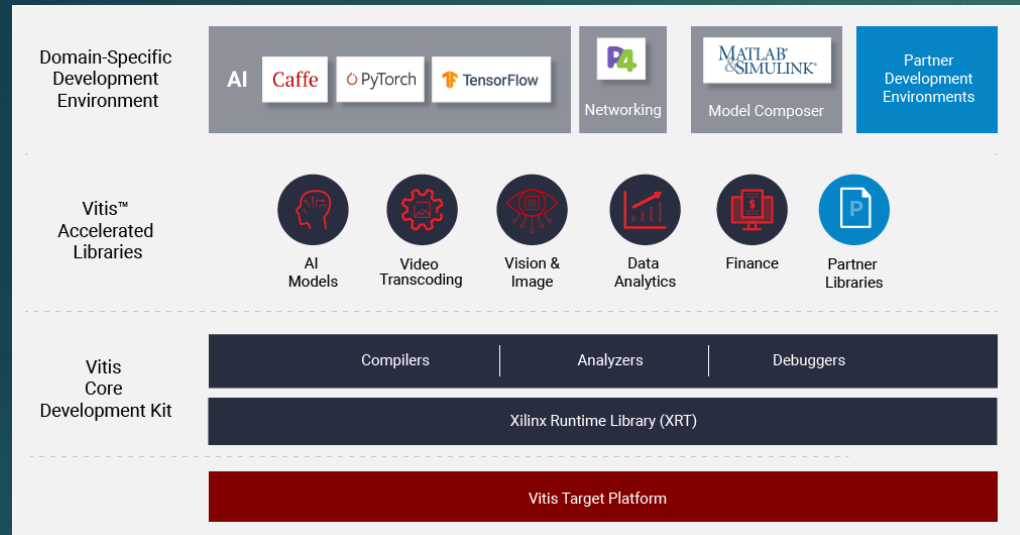
AMD CORPORATE TEMPLATE | 2018

AMD.com/HPC

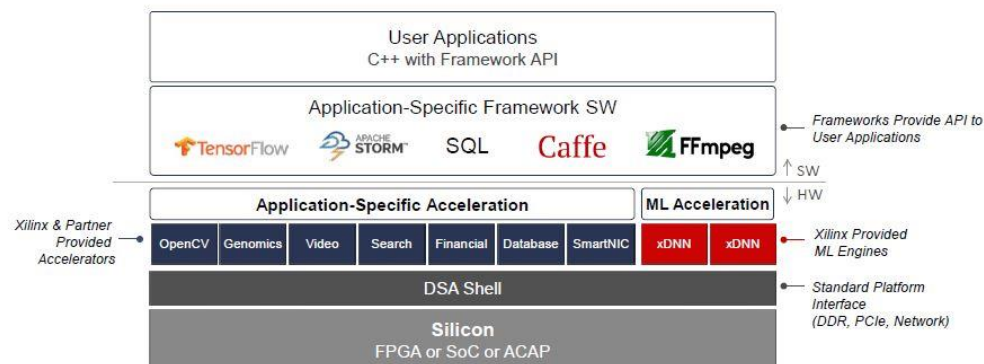


RADEON

Xilinx Framework



Stack for Application Acceleration including ML

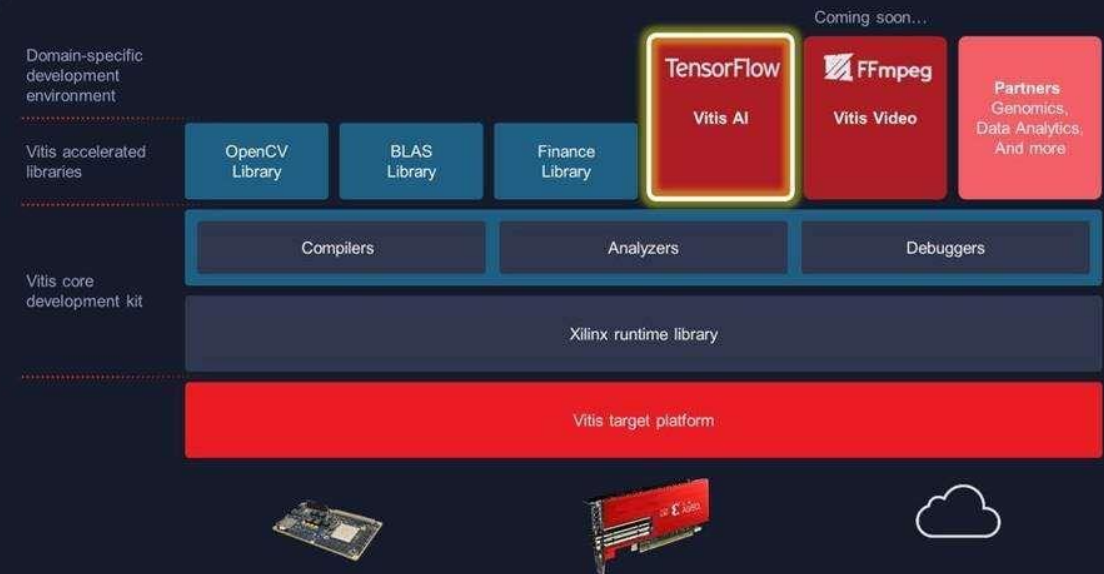


Adaptable For Any Application

Software Programmable

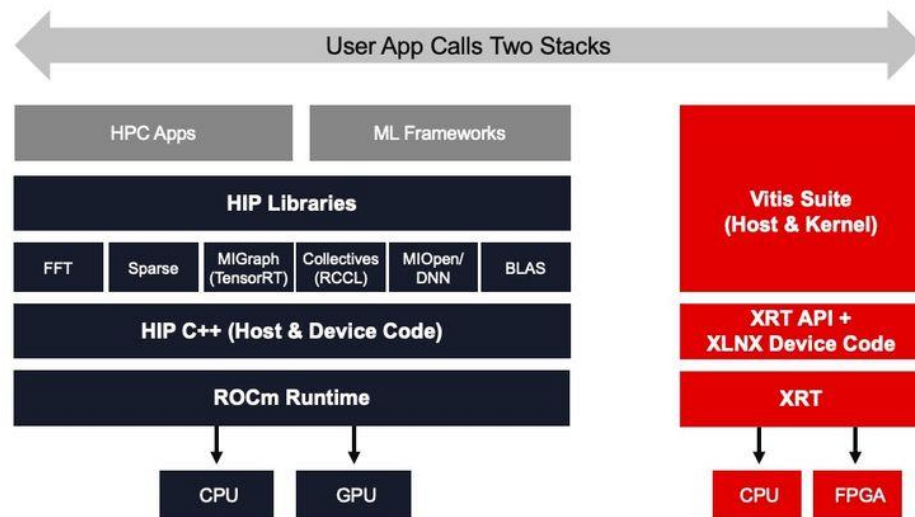
Heterogeneous Platform

Vitis: Unified Software Platform

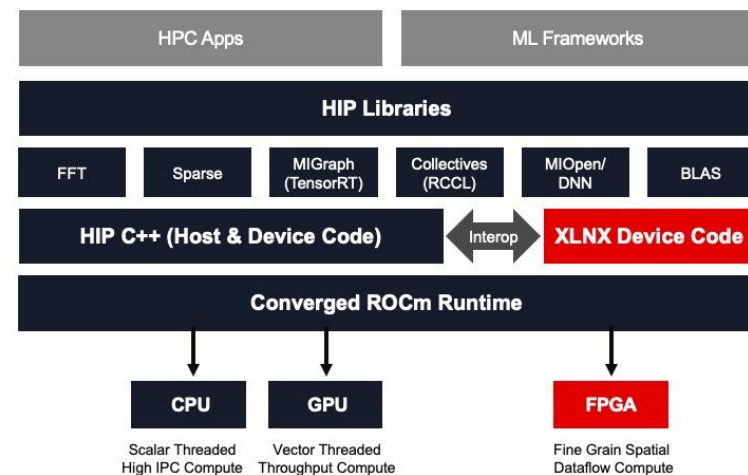


AMD and Xilinx Integration Framework 16

Current: Two Stacks

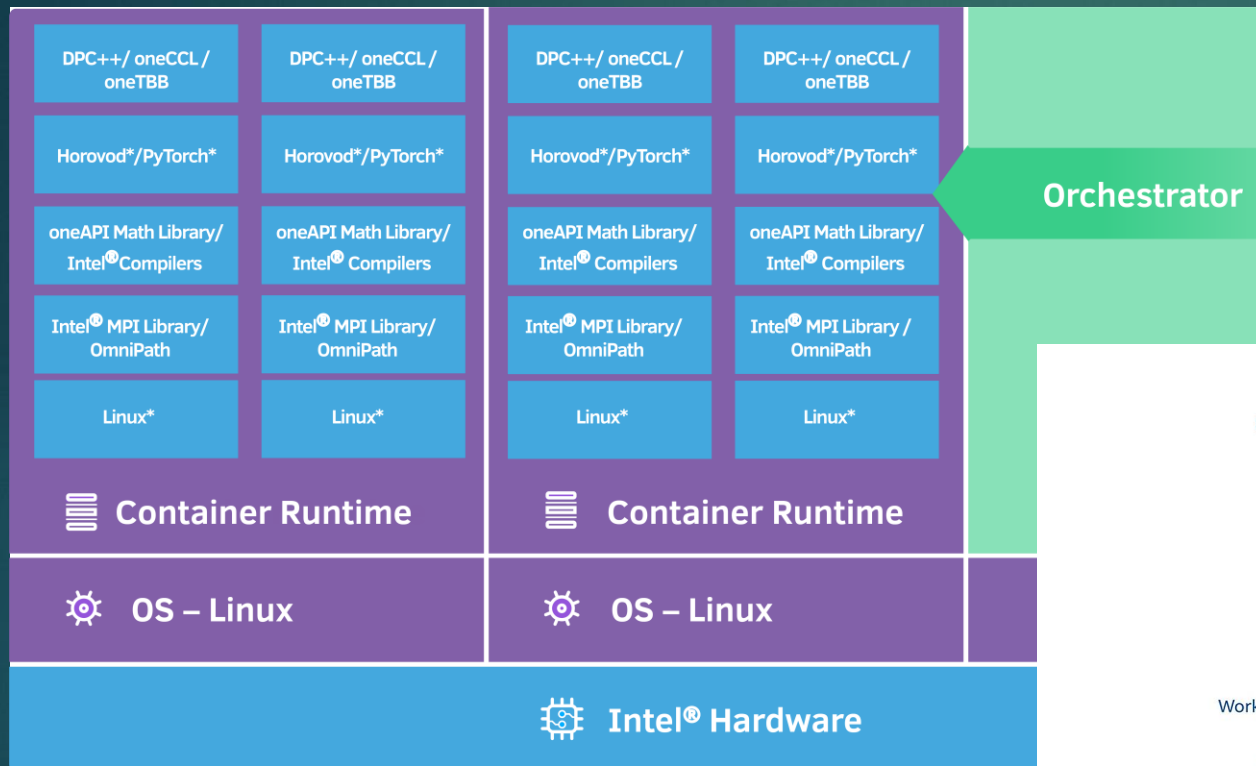


Preview: ROCm Runtime Integration



Intel Framework

17

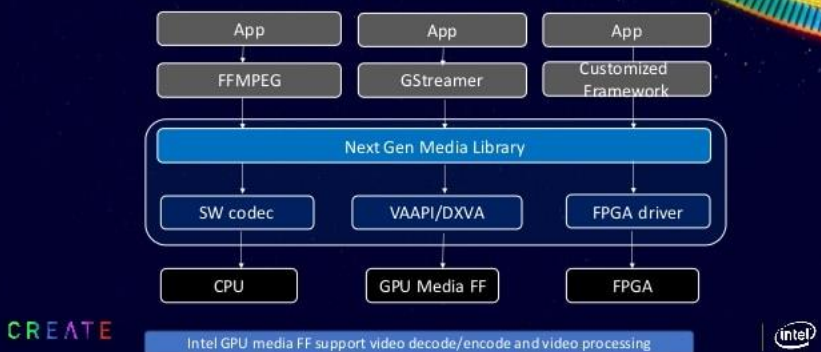


oneCCL: oneAPI Collective Communications Library
oneTBB: oneAPI Threading Building Blocks

Intel® software

Infrastructure

Intel Video software Offering



Dynamically allocate Intel® FPGAs for workload optimization

Enhanced end user experience

Focus on system architecture

Fast-track your performance

Workload optimization with less effort

Common developer interface for Intel® FPGA data center products

Decrease server TCO, simplify management

Rack-level solutions

User applications

Industry standard software frameworks

Acceleration libraries

Intel® developer tools

(Intel® Quartus® Prime, Intel® FPGA SDK for OpenCL™, Intel® Parallel Studio XE)

Acceleration environment

(Intel® Acceleration Engine with OPAE technology, FPGA interface manager (FIM))

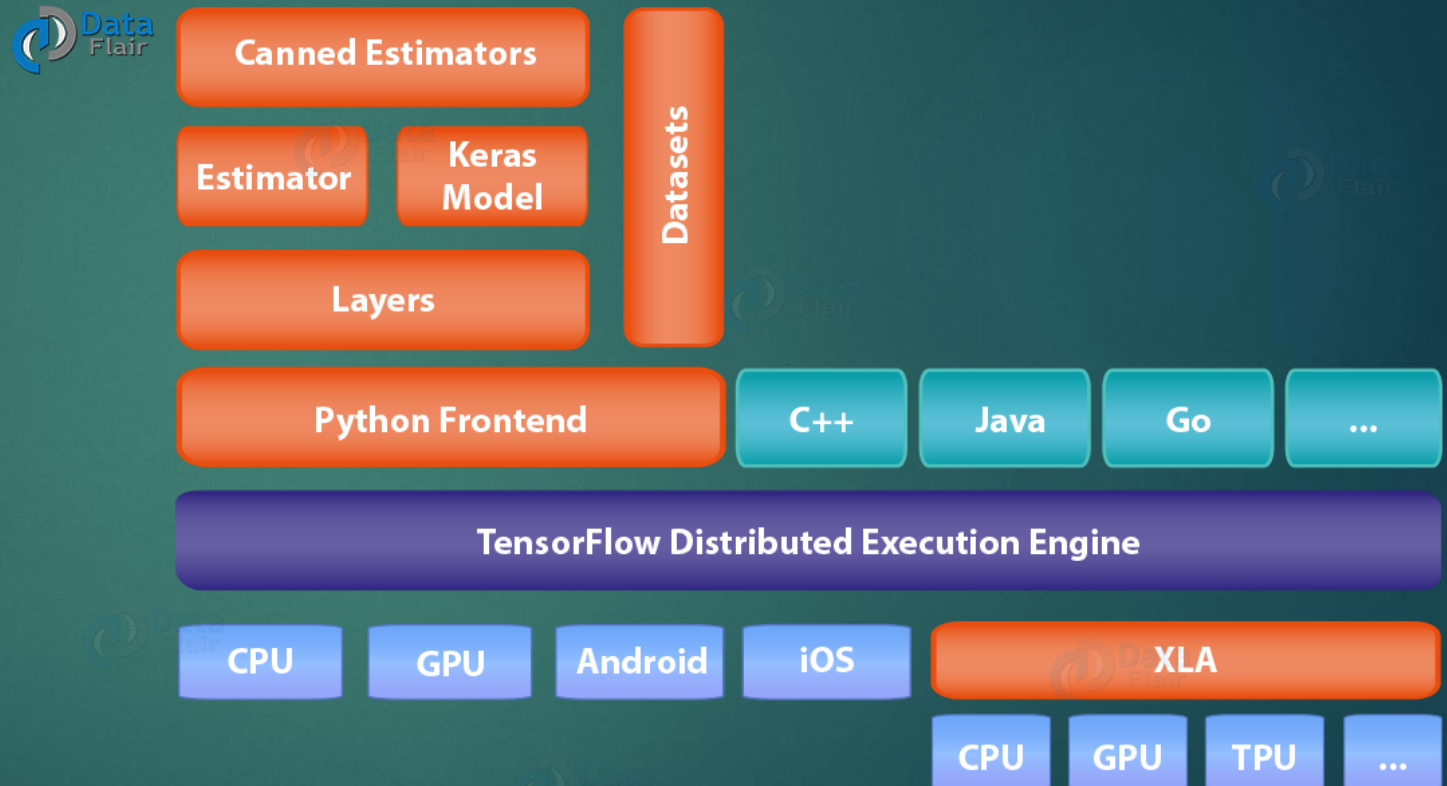
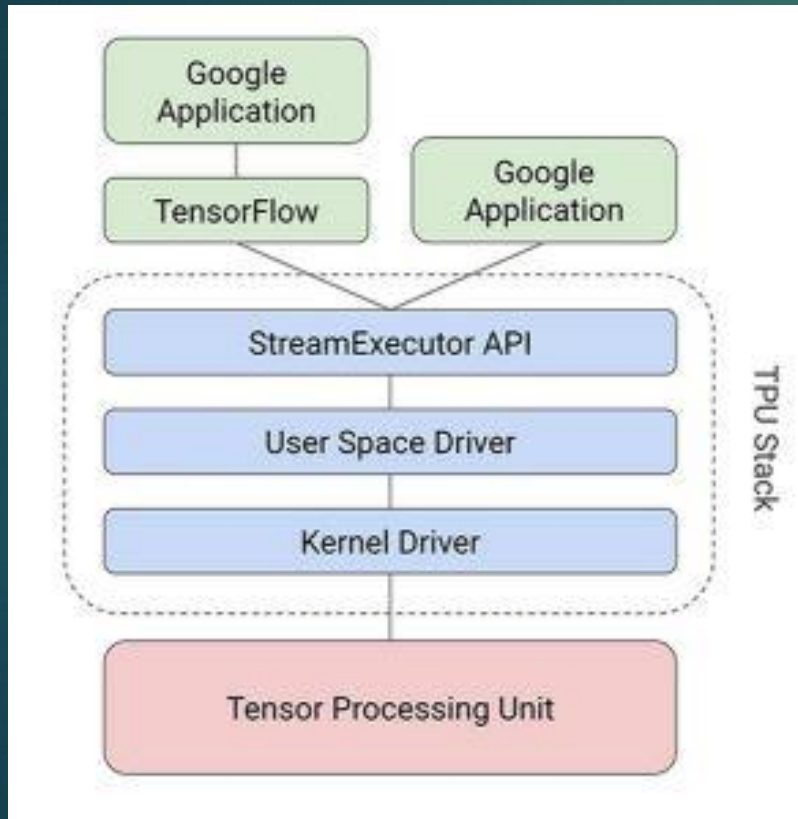
OS and virtualization environment



OpenCL and the OpenCL logo are trademarks of Apple Inc. used by permission by Khronos. Some names pending final approval and may change in the future.

TPU / TensorFlow Framework

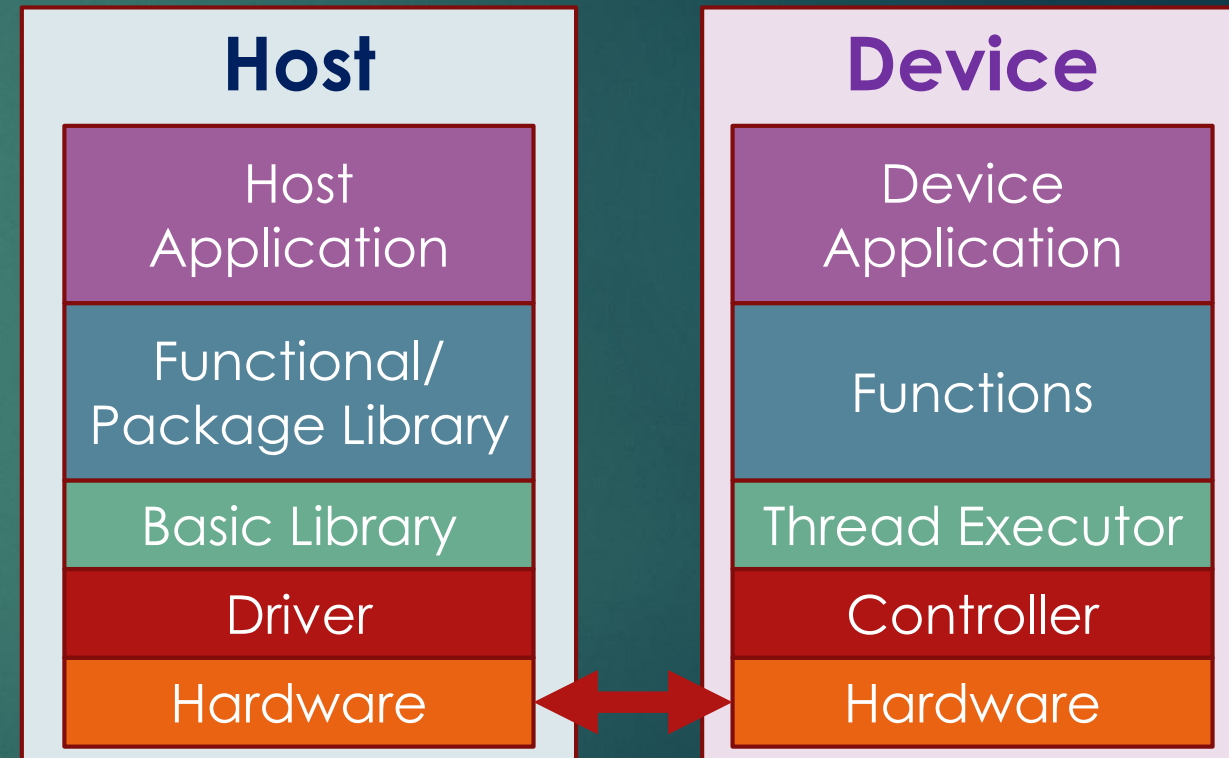
18



Computing Software Stack

19

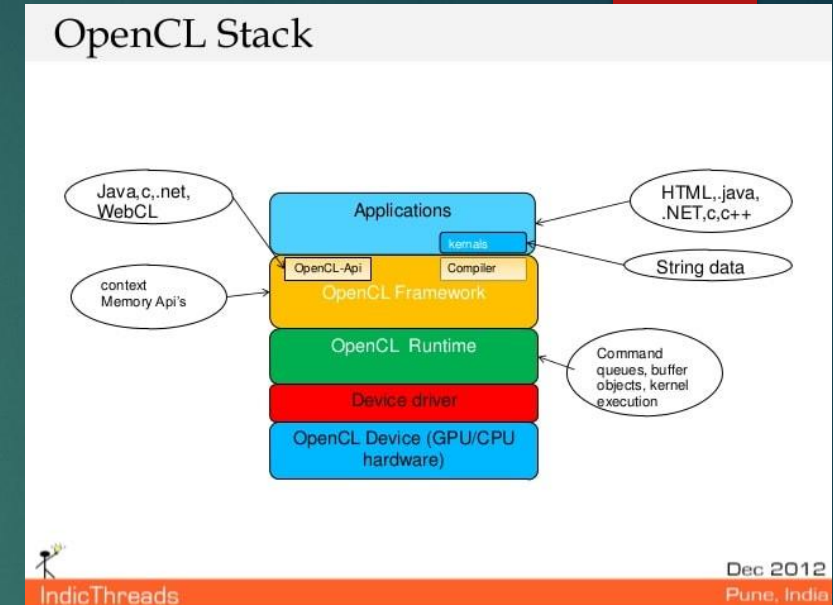
- ▶ Host can have multiple Devices
- ▶ Hardware Interfaces
- ▶ Software Interfaces by API
 - ▶ Application Programming Interfaces
- ▶ Device Execution/Operations controlled by Host (API)
- ▶ Depend on Host OS / Device



Conclusion

20

- ▶ Computing Codes => 2 sides
 - ▶ Host => Control/Manage Devices
 - ▶ Devices => Execute
- ▶ Code Development
 - ▶ Source > Intermediate > Executable Codes
- ▶ Code Execution
 - ▶ Place and Execute Codes
 - ▶ Executed at Devices
 - ▶ Controlled by Host
- ▶ Code Verification
- ▶ Many Platforms/Frameworks



TensorFlow on Cuda		TensorFlow on Open Standards	
Python	C++	Python	C++
TensorFlow C API		TensorFlow C API	
TF Kernels (800+)	Matrix Multiply	TF Kernels (800+)	Matrix Multiply
Eigen Tensors	cuDNN	Eigen Tensors	SYCL - OpenCL
Cuda C++	Hand-coded Cuda	SYCL	Hand-coded OpenCL
Nvidia GPU		OpenCL	
		Any CPU, GPU, FPGA, etc	

END

QUESTIONS & ANSWERS